

LAB 4 PROMPTS

SRN: PES2UG24CS437

Name: Samyuktha Ramesh Babu

Section: H

Starting prompt

I am a game developer. I have a fishing game with:

- A boat and hook - the hook casts downward and retracts back to the surface on its own, in a continuous loop
- Fish that swim back and forth across the pond at fixed depths and wrap around the screen edges
- A hook that snags a fish on contact - the fish rides up attached to the hook and is only actually caught (scored, removed from the pond) once the hook fully returns to the surface
- A running score display

I want to fix 1 bug and add 3 features to this fishing game. Currently, the game has no controls. I want you to implement these features after I give you the task and the source file corresponding to it.

Task 1

Task 1: Fix the catch-detection bug

A fish is supposed to be caught only when the hook actually overlaps it. In the current build, `check_catch` (in `game/catch.py`) only compares the hook's depth to the fish's depth, within a fixed 10-pixel tolerance - it never compares horizontal position at all. That means a fish can register as caught while it's still far off to one side of the screen, just because the hook happens to be passing through its depth. Fix the check so it's based on the hook and fish actually overlapping, not just being at a similar depth.

game/catch.py contents are:

```
"""
```

```
catch: hook-vs-fish catch detection.
```

```
"""
```

```
def check_catch(hook, fish_list):
```

```
    """
```

```
    Returns the fish the hook has caught, or None.
```

```
    """
```

```
    for fish in fish_list:
```

```
        if abs(hook.y - fish.y) < 10:
```

```
            return fish
```

```
    return None
```

game/fish.py contents are:

"""

Fish: swims horizontally at a fixed depth, wrapping around when it exits the screen. The starter has one fish type; Task 2 adds more.

"""

```
import pygame
```

```
class Fish:
```

```
    def __init__(self, x, y, speed, width=36, height=18, point_value=10, color=(80, 180, 220)):
```

```
        self.x = float(x)
```

```
        self.y = y
```

```
        self.speed = speed
```

```
        self.width = width
```

```
        self.height = height
```

```
        self.point_value = point_value
```

```
        self.color = color
```

```
    def update(self, screen_width):
```

```
        self.x += self.speed
```

```
        if self.speed > 0 and self.x > screen_width:
```

```
            self.x = -self.width
```

```
        elif self.speed < 0 and self.x < -self.width:
```

```
            self.x = screen_width
```

```
    def get_rect(self):
```

```
        return pygame.Rect(
```

```
            int(self.x - self.width / 2), int(self.y - self.height / 2),
```

```
            self.width, self.height,
```

```
        )
```

game/game_engine.py contents are:

"""

GameEngine: owns the hook and the fish, and runs one frame's worth of game logic.

Starter version: the hook casts and retracts automatically in a continuous loop - there's no player control over casting yet (that's Task 3), only one fish type exists (Task 2 adds more), and there's no round timer (Task 4). Catch detection also has a known bug (see game/catch.py) that Task 1 asks you to fix.

"""

```
from game.hook import Hook, IDLE
from game.fish import Fish
from game.catch import check_catch
from game.renderer import WIDTH, HEIGHT, SURFACE_Y, MAX_DEPTH_Y
```

```
class GameEngine:
    def __init__(self):
        self.hook = Hook(x=WIDTH / 2, surface_y=SURFACE_Y, max_depth_y=MAX_DEPTH_Y,
speed=5)
        self.fish_list = [
            Fish(x=100, y=180, speed=2, point_value=10, color=(80, 180, 220)),
            Fish(x=400, y=280, speed=-2, point_value=10, color=(80, 180, 220)),
            Fish(x=250, y=380, speed=3, point_value=10, color=(80, 180, 220)),
        ]
        self.hooked_fish = None
        self.score = 0

    def update(self):
        if self.hook.state == IDLE:
            self.hook.start_cast()

        self.hook.update()

        for fish in self.fish_list:
            fish.update(WIDTH)

        if self.hooked_fish is not None:
            self.hooked_fish.x = self.hook.x
            self.hooked_fish.y = self.hook.y
            if self.hook.state == IDLE:
                self.score += self.hooked_fish.point_value
                self.hooked_fish = None
        else:
            caught = check_catch(self.hook, self.fish_list)
            if caught is not None:
                self.fish_list.remove(caught)
                self.hooked_fish = caught
                self.hooked_fish.x = self.hook.x
                self.hooked_fish.y = self.hook.y
                self.hook.catch_fish()

    def draw(self, surface, font):
        from game import renderer
```

```

draw_list = list(self.fish_list)
if self.hooked_fish is not None:
    draw_list.append(self.hooked_fish)
renderer.draw_scene(surface, self.hook, draw_list)
renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))

```

game/hook.py contents are:

```

"""

```

Hook: the player's fishing hook. Moves only vertically, at a fixed horizontal position. In the starter, it casts and retracts on its own, in a continuous loop, with no player input at all - Task 3 changes this to be player-controlled.

```

"""

```

```

import pygame

```

```

IDLE = "idle"

```

```

CASTING = "casting"

```

```

RETRACTING = "retracting"

```

```

class Hook:

```

```

    def __init__(self, x, surface_y, max_depth_y, speed=4, width=14, height=14):

```

```

        self.x = x

```

```

        self.surface_y = surface_y

```

```

        self.max_depth_y = max_depth_y

```

```

        self.y = surface_y

```

```

        self.speed = speed

```

```

        self.width = width

```

```

        self.height = height

```

```

        self.state = IDLE

```

```

    def start_cast(self):

```

```

        self.state = CASTING

```

```

        self.y = self.surface_y

```

```

    def update(self):

```

```

        if self.state == CASTING:

```

```

            self.y += self.speed

```

```

            if self.y >= self.max_depth_y:

```

```

                self.y = self.max_depth_y

```

```

                self.state = RETRACTING

```

```

            elif self.state == RETRACTING:

```

```

                self.y -= self.speed

```

```
    if self.y <= self.surface_y:
        self.y = self.surface_y
        self.state = IDLE
```

```
def catch_fish(self):
    """Called when a fish has been caught - immediately head back up."""
    self.state = RETRACTING
```

```
def get_rect(self):
    return pygame.Rect(
        int(self.x - self.width / 2), int(self.y - self.height / 2),
        self.width, self.height,
    )
```

game/renderer.py contents are:

```
"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""
```

```
import pygame
```

```
WIDTH, HEIGHT = 700, 500
SURFACE_Y = 80
MAX_DEPTH_Y = HEIGHT - 40
```

```
WINDOW_SIZE = (WIDTH, HEIGHT)
```

```
COLOR_SKY = (140, 200, 230)
COLOR_WATER = (30, 90, 150)
COLOR_BOAT = (120, 80, 50)
COLOR_LINE = (240, 240, 240)
COLOR_HOOK = (220, 220, 220)
COLOR_TEXT = (255, 255, 255)
```

```
def draw_scene(surface, hook, fish_list):
    surface.fill(COLOR_SKY, pygame.Rect(0, 0, WIDTH, SURFACE_Y))
    surface.fill(COLOR_WATER, pygame.Rect(0, SURFACE_Y, WIDTH, HEIGHT -
SURFACE_Y))
```

```
    pygame.draw.rect(surface, COLOR_BOAT, (hook.x - 40, SURFACE_Y - 20, 80, 22))
```

```
    pygame.draw.line(surface, COLOR_LINE, (hook.x, SURFACE_Y), (hook.x, hook.y), 2)
    pygame.draw.circle(surface, COLOR_HOOK, (int(hook.x), int(hook.y)), 7)
```

```
for fish in fish_list:
    pygame.draw.ellipse(surface, fish.color, fish.get_rect())
```

```
def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)
```

```
def draw_banner(surface, font, text):
    surf = font.render(text, True, (255, 220, 80))
    rect = surf.get_rect(center=(surface.get_width() // 2, surface.get_height() // 2))
    surface.blit(surf, rect)
```

Task 2

Task 2: Implement multiple fish types

Introduce at least two fish types that differ in movement speed and point value (for example, a slow low-value fish and a fast high-value one). Give each type its own look (color and/or size) so they're visually distinguishable, and make sure the correct point value is awarded when each type is caught.

Task 3

Task 3: Implement player-controlled casting

Change the hook so the player decides when it casts, instead of it looping automatically. Pressing a key should start a cast if the hook is currently idle; it should then travel down and return on its own (already handled) once it reaches maximum depth or catches a fish. A new cast should not be able to interrupt one that's already in progress.

Task 4

Task 4: Implement a 30-second round timer

Add a 30-second countdown for the round. Display the remaining time on screen. Once it reaches zero, no further catches should be possible, the round should end, and the final score should be shown clearly. Provide a way to start a new round with the score and timer both reset.

Bug fixing

This is the contents of [main.py](#) which I had forgotten to give you, can you fix it?

```
"""
```

Fishing Game (Lab Starter)

Run with: python3 main.py

```
"""
```

```
import pygame
```

```

from game.game_engine import handle_input
from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE

def main():
    pygame.init()
    screen = pygame.display.set_mode(WINDOW_SIZE)
    pygame.display.set_caption("Fishing")
    clock = pygame.time.Clock()
    font = pygame.font.SysFont("consolas", 22)

    engine = GameEngine()
    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            handle_input(event)

        engine.update()
        engine.draw(screen, font)

        pygame.display.flip()
        clock.tick(60)

    pygame.quit()

if __name__ == "__main__":
    main()

```

The timer isn't working, it's not visible